PUC Minas

LAB01	Laboratório 01 - 15 pontos

INFORMAÇÕES DOCENTE						
CURSO: ENGENHARIA DE SOFTWARE	DISCIPLINA: LABORATÓRIO DE EXPERIMENTAÇÃO DE SOFTWARE	TURNO	MANHÃ	TARDE	NOITE	PERÍODO/SALA: 6º
					x	
ALUNOS (A): Leonardo Vieira Machado e Nicolas Almeida Prado da Silva						

Características de repositórios populares

1. Introdução

1.1 Contextualização

Plataformas de hospedagem de código-fonte, como o GitHub, tornaram-se vastos repositórios de dados sobre o desenvolvimento de software moderno. Projetos de código aberto (Open Source) hospedados nessas plataformas variam enormemente em tamanho, atividade e popularidade. Compreender as características que tornam um repositório "popular" medido pelo número de *stars* (estrelas) pode fornecer *insights* valiosos sobre boas práticas de engenharia de software, engajamento da comunidade e manutenção de sistemas.

1.2 Problema foco do experimento

O problema central deste experimento é a falta de clareza empírica sobre o que caracteriza um software de sucesso e alta popularidade no GitHub. O experimento busca investigar se repositórios altamente populares compartilham atributos em comum, como maturidade (idade), nível de colaboração externa, frequência de atualizações e linguagem de programação utilizada.

1.3 Questões-Pesquisa (RQs)

Para guiar o experimento, foram definidas as seguintes Questões de Pesquisa:

- RQ 01. Sistemas populares são maduros/antigos?
- RQ 02. Sistemas populares recebem muita contribuição externa?
- RQ 03. Sistemas populares lançam releases com frequência?
- RQ 04. Sistemas populares são atualizados com frequência?
- RQ 05. Sistemas populares são escritos nas linguagens mais populares?
- RQ 06. Sistemas populares possuem um alto percentual de issues fechadas?

1.4 Hipótese(s)

- H1: Sistemas populares tendem a ser mais antigos (maduros), pois acumulam usuários e reputação ao longo do tempo.
- H2: Sistemas populares possuem um alto índice de contribuição externa (Pull Requests de não-membros) e uma alta taxa de resolução de problemas (issues fechadas), refletindo comunidades ativas.
- H3: Repositórios populares são atualizados com alta frequência (baixo tempo desde a última atualização) e possuem um número expressivo de releases.

1.5 Objetivo

- Objetivo Principal: Analisar empírica e quantitativamente as características dos 1000 repositórios mais populares do GitHub para responder às questões de pesquisa propostas.
- Objetivos Específicos: 1. Coletar dados via API GraphQL do GitHub de forma automatizada. 2. Calcular métricas específicas de tempo, contribuição e resolução de problemas. 3. Exportar os dados processados para um formato tabular (.csv) para facilitar a análise de dados posterior.

2. Metodologia

2.1 Passo a passo do experimento

1. Configuração do Ambiente: Criação de um ambiente virtual Python isolado e configuração de um Token de Acesso Pessoal (PAT) do GitHub para autenticação.
2. Desenvolvimento do Script de Coleta: Implementação de um script em Python para realizar requisições à API GraphQL do GitHub.
3. Execução da Coleta: Busca automatizada dos 1000 repositórios com maior número de estrelas (stars:>10000 sort:stars-desc), utilizando paginação.
4. Processamento das Métricas: Cálculo das idades em dias, contagem de Pull Requests externos aceitos, proporção de issues fechadas e tempo desde a última atualização.
5. Exportação: Salvamento dos dados processados em um arquivo relatorio_repositorios.csv.
6. Análise: (Este passo você fará com os dados gerados, analisando as planilhas).

2.2 Decisões

- Uso de GraphQL em vez de REST: Optou-se pela API GraphQL para evitar o problema de *Over-fetching* (trazer dados desnecessários) e *Under-fetching* (fazer múltiplas requisições para o mesmo repositório). Isso reduziu drasticamente o tempo de coleta e o risco de bloqueio por limite de requisições.
- Paginação e Tratamento de Erros (Retry): Devido ao volume de dados (como consultar as *issues* e PRs de 1000 grandes projetos), o servidor do GitHub ocasionalmente retornava erro 502 (Bad Gateway). A decisão arquitetural foi implementar consultas menores (lotes de 10 em 10) atreladas a uma lógica de *Exponential Backoff* (tentativas repetidas com pausas progressivas) para garantir a resiliência do script sem interrupção manual.

2.3 Materiais utilizados

- Linguagem: Python

- IDE: Visual Studio Code (VS Code).
- Bibliotecas Python: requests (para comunicação HTTP), python-dotenv (para gerenciamento seguro do token), datetime (para cálculos de tempo) e csv (para exportação).
- Fonte de Dados: GitHub GraphQL API.

2.4 Métodos utilizados

O experimento utiliza o método de Mineração de Repositórios de Software. Trata-se de uma abordagem quantitativa e empírica, onde dados secundários gerados organicamente pela plataforma GitHub são extraídos de forma automatizada e tratados estatisticamente para identificar padrões.

2.5 Métricas e suas Unidades

- Idade do repositório (RQ 01): Unidade: Dias inteiros. Calculado pela diferença entre a data atual e a data de createdAt.
- Contribuição externa (RQ 02): Unidade: Contagem (inteiro). Número de Pull Requests com status MERGED onde o autor não é OWNER, MEMBER ou COLLABORATOR.
- Frequência de releases (RQ 03): Unidade: Contagem (inteiro). Representado pelo totalCount do nó de *releases*.
- Tempo até a última atualização (RQ 04): Unidade: Dias inteiros. Calculado pela diferença entre a data atual e a data de updatedAt.
- Linguagem popular (RQ 05): Unidade: Categórica (String). Nome da primaryLanguage identificada pelo GitHub.
- Percentual de issues fechadas (RQ 06): Unidade: Porcentagem (%). Razão entre o total de issues CLOSED e o total geral de issues, multiplicada por 100.

3. Visualização dos Resultados

3.1 Visualização tabular e gráfica dos dados colhidos

A extração de dados retornou *um data*set composto por 1000 registros, correspondentes aos repositórios mais populares do GitHub no momento da coleta. Devido ao volume de dados gerados no arquivo bruto (.csv), optou-se por apresentar os resultados de forma sumarizada, utilizando visualizações gráficas e tabelas agregadas. Essa abordagem permite uma interpretação direta das métricas relacionadas às Questões de Pesquisa (RQs) estabelecidas na metodologia.

Abaixo, apresentam-se as visualizações que ilustram o perfil dos sistemas populares analisados:

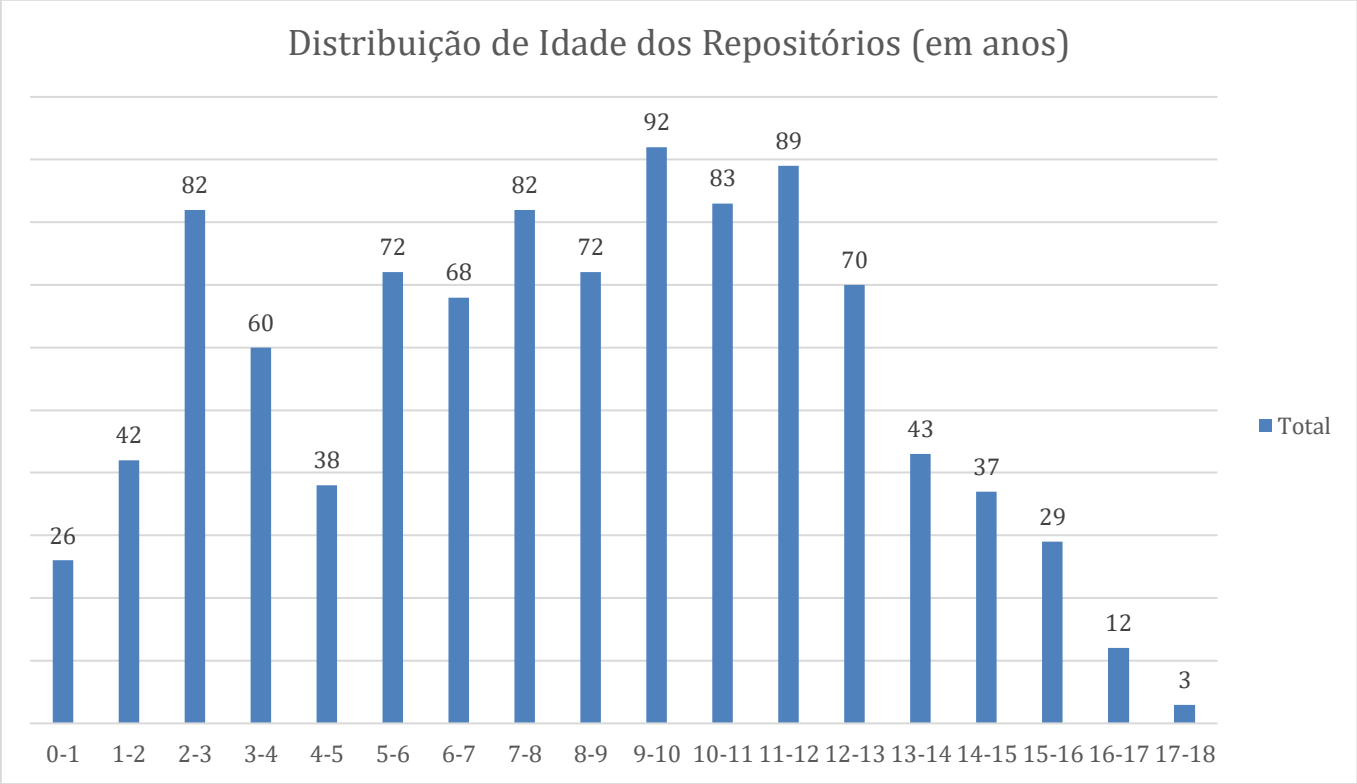

Gráfico 1: Distribuição de Idade dos Repositórios (em anos).

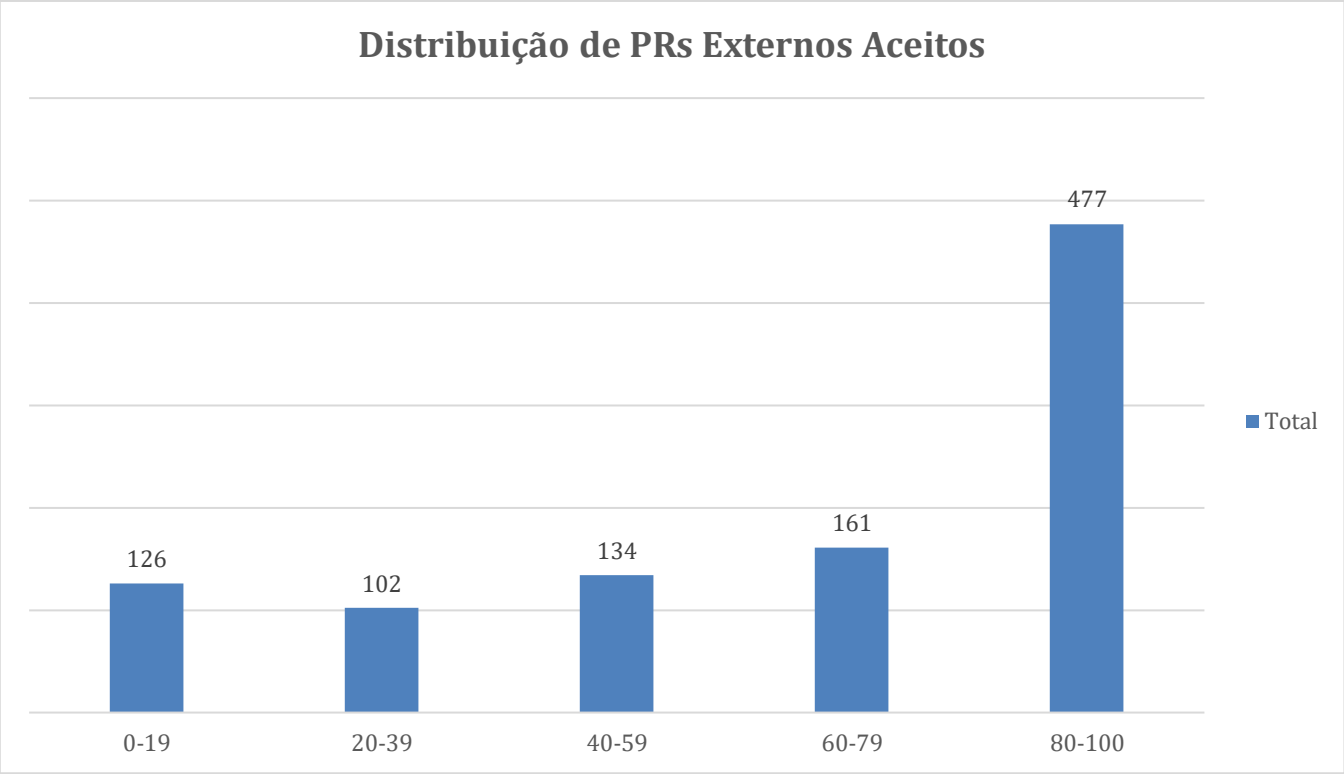

Gráfico 2: Distribuição de PRs Externos Aceitos.

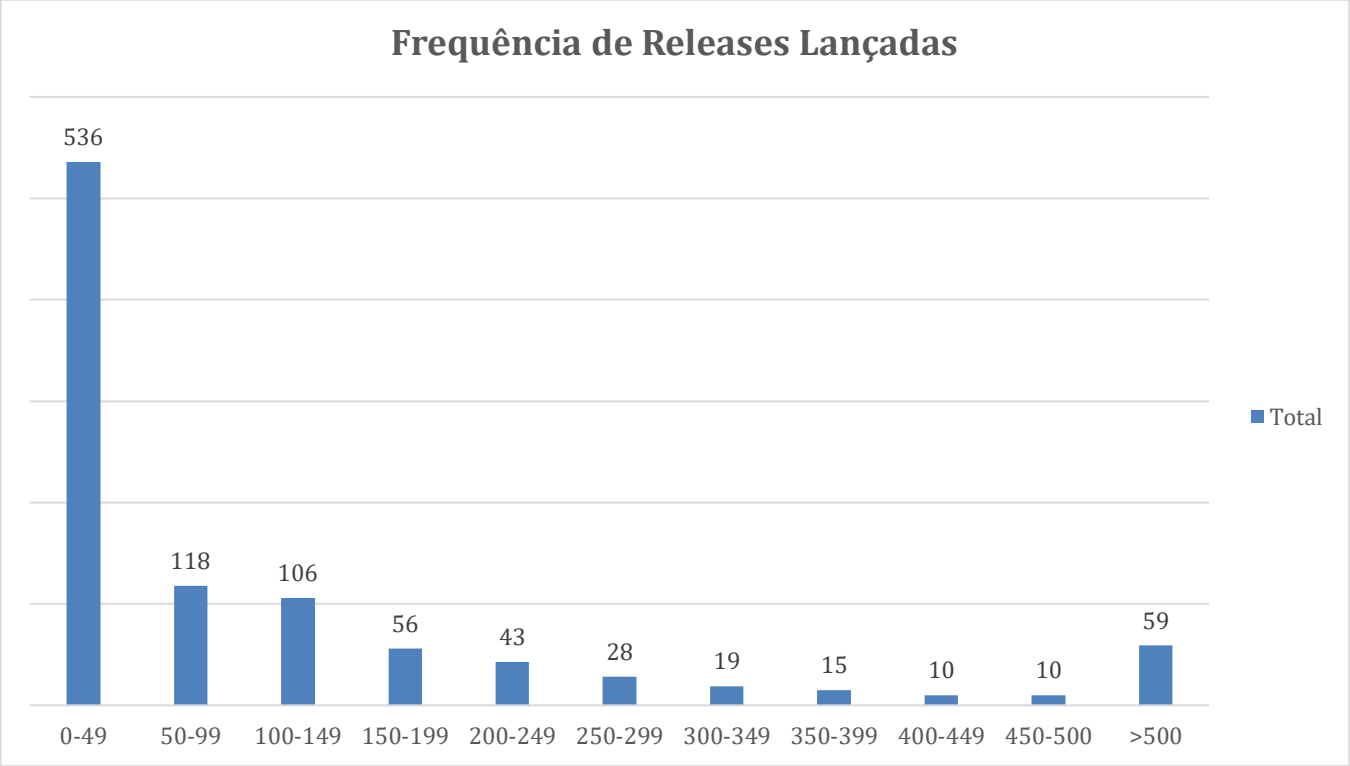

Gráfico 3: Frequência de Releases Lançadas.

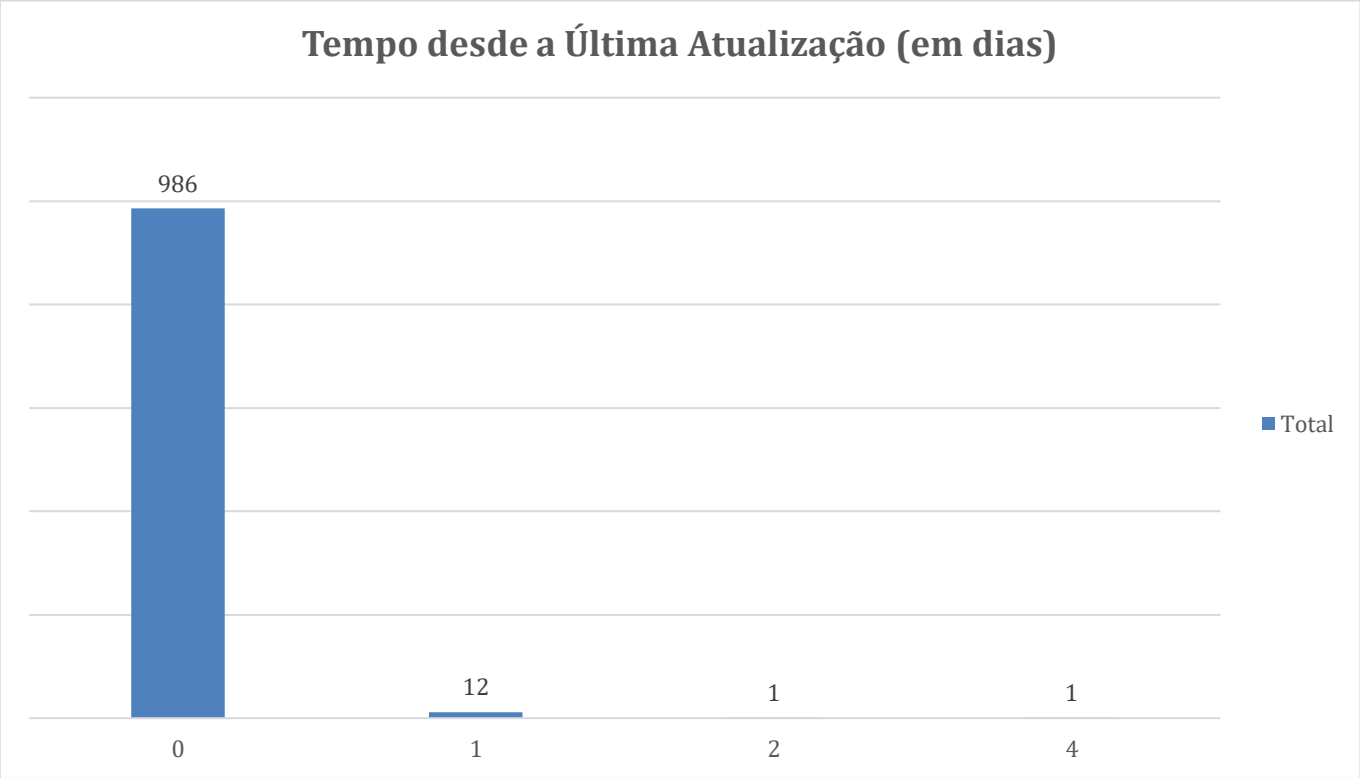

Gráfico 4: Tempo desde a Última Atualização (em dias)

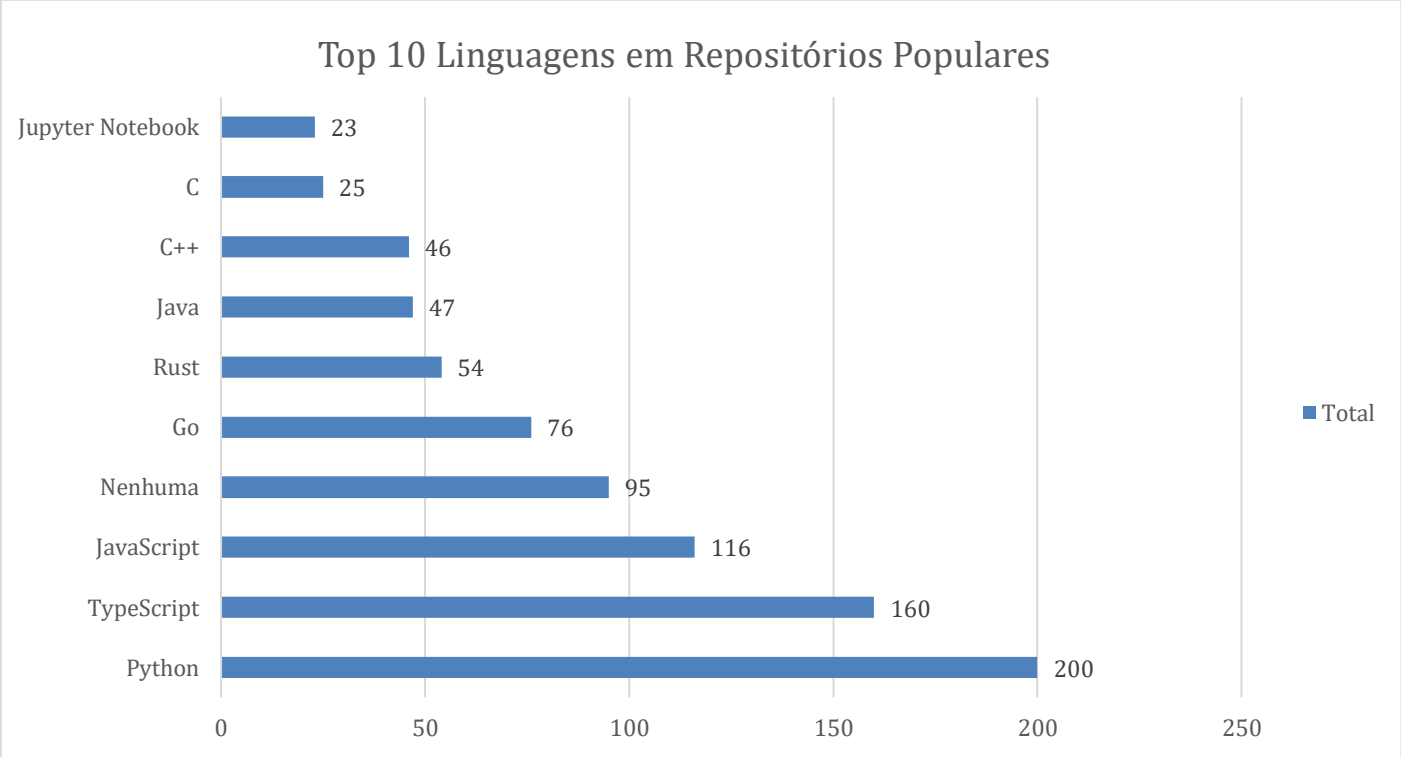

Gráfico 5: Top 10 Linguagens em Repositórios Populares.

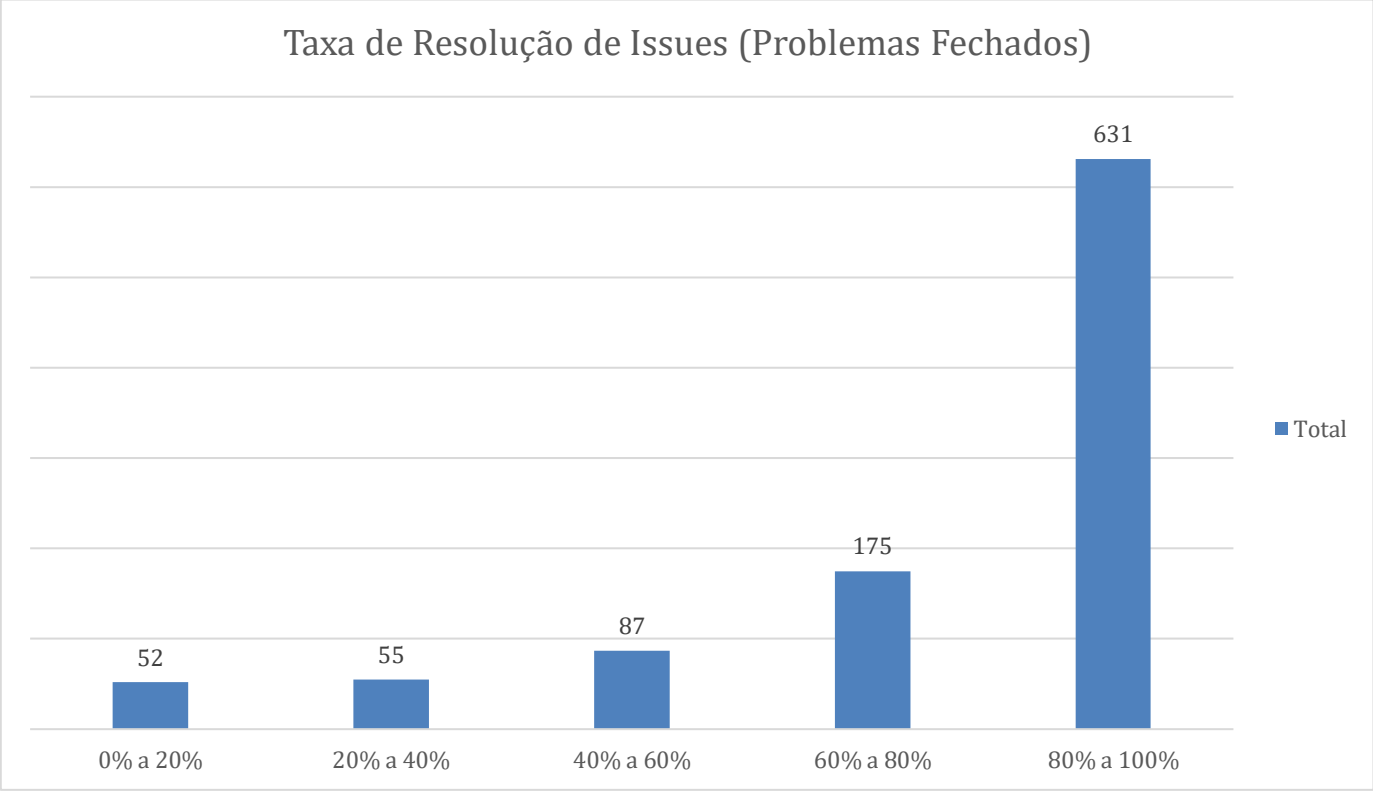

Gráfico 6: Taxa de Resolução de Issues (Problemas Fechados).

4. Discussão dos Resultados

Nesta seção, as Questões de Pesquisa (RQs) são confrontadas com os dados coletados para validar as hipóteses levantadas.

4.1 Confronto das Questões de Pesquisa

- **RQ 01. Sistemas populares são maduros/antigos?**
  - **Análise:** O **Gráfico 1** revela que a grande maioria dos repositórios possui entre **7 e 13 anos**. Há uma queda expressiva em projetos com menos de 2 anos.
  - **Veredito:** A **H1 é confirmada**. A popularidade no GitHub não é instantânea; ela parece ser um subproduto da maturidade e do tempo de exposição na comunidade.
- **RQ 02. Sistemas populares recebem muita contribuição externa?**
  - **Análise:** O **Gráfico 2** mostra que quase metade da amostra (477 repositórios) possui o nível máximo de PRs externos analisados (80-100).
  - **Veredito:** A **H2 é confirmada**. Projetos com mais de 10.000 estrelas mantêm comunidades altamente colaborativas, sugerindo que a abertura para contribuições externas é um motor de crescimento.
- **RQ 03. Sistemas populares lançam releases com frequência?**
  - **Análise:** O **Gráfico 3** indica um comportamento inesperado: a maioria dos projetos (536) possui entre 0 e 49 releases. Embora existam "outliers" com mais de 500 releases, a frequência não é uniformemente alta.
  - **Veredito:** A **H3 é parcialmente refutada**. A popularidade não exige necessariamente um volume massivo de releases formais, podendo focar em estabilidade ou entrega contínua sem tags de versão.
- **RQ 04. Sistemas populares são atualizados com frequência?**
  - **Análise:** O **Gráfico 4** é contundente: **986 dos 1000 repositórios** foram atualizados no dia da coleta (0 dias sem atualizar).
  - **Veredito:** A **H3 é confirmada** neste quesito. A manutenção ativa e constante é a característica mais homogênea entre os sistemas populares.
- **RQ 05. Sistemas populares são escritos nas linguagens mais populares?**
  - **Análise:** O **Gráfico 5** posiciona **Python (200)**, **TypeScript (160)** e **JavaScript (116)** no topo. Essas linguagens dominam o ecossistema de desenvolvimento moderno.
  - **Veredito:** Hipótese confirmada. Há uma correlação direta entre o uso de linguagens com grandes ecossistemas e a popularidade do repositório.
- **RQ 06. Sistemas populares possuem um alto percentual de issues fechadas?**
  - **Análise:** Segundo o **Gráfico 6**, mais de 600 repositórios fecham entre **80% e 100%** de suas issues.
  - **Veredito:** A **H2 é confirmada**. O sucesso de um projeto está atrelado à sua capacidade de

resposta e resolução de problemas reportados pela comunidade.

4.2 Insights e Estatísticas

A partir da análise, percebe-se que a **"fórmula da popularidade"** no GitHub não depende de um único fator, mas de um ecossistema saudável:

1. **Atividade Contínua:** Quase 100% dos projetos são "vivos" (atualizações diárias).
2. **Linguagem como Porta de Entrada:** Python e TypeScript facilitam o engajamento inicial.
3. **Eficiência Operacional:** O alto índice de fechamento de issues demonstra uma gestão de projeto profissional, mesmo em ambientes voluntários.

5. Conclusão

Este experimento permitiu concluir que a popularidade de um repositório no GitHub (medida por estrelas) é um indicador de **confiabilidade e manutenção ativa**, mais do que apenas inovação momentânea.

5.1 Tomada de Decisão

Para desenvolvedores ou empresas que desejam criar projetos de sucesso, os dados sugerem:

- Priorizar a **manutenção constante** em vez de lançamentos esporádicos.
- Fomentar a **contribuição externa**, mantendo um fluxo eficiente de aceitação de PRs.
- Garantir uma **taxa de resolução de issues superior a 80%** para manter a confiança da comunidade.

5.2 Sugestões Futuras

Para trabalhos futuros, recomenda-se a análise de outros requisitos, podendo cruzandar o impacto da linguagem especificamente sobre a taxa de contribuição externa, além de investigar se o uso de ferramentas de CI/CD (GitHub Actions) influencia na frequência de releases.